

Final Report · Mini Project

SeeSh: See-Through Shell for Visualizing System Calls

ผู้จัดทำ:

นายอาณัฐ อารีย์	673380432-5
นายพัทธคนย์ คำนัน	673380416-3
นายณัฐภัทร ฉ่ำตะคุ	673380583-4

รายวิชา: Operating Systems and System Calls Programming มหาวิทยาลัยขอนแก่น

Source code: <https://github.com/Arnat-Aree/seesh>

1. ภาพรวม

SeeSh คือ shell ที่เราเขียนขึ้นเองด้วยภาษา C โดยใช้ POSIX system call จริง (The Open Group, 2018) ใช้งานได้เหมือน shell ทั่วไป และทุกครั้งที่รันคำสั่ง SeeSh จะส่งข้อมูลว่าตัวเองเรียก system call อะไรบ้างไปแสดงบนหน้าเว็บ ซึ่งอธิบายการทำงานที่ละขั้นเป็นภาษาไทย

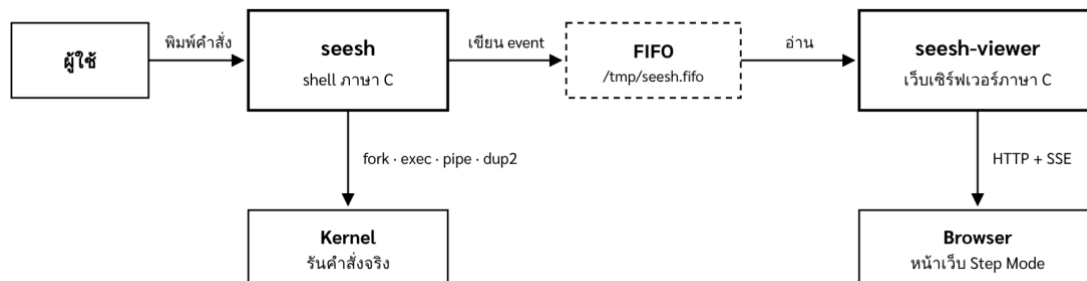
ที่มาก็คือ เวลาพิมพ์คำสั่งอย่าง `ls | wc -l` เราเห็นแค่ผลลัพธ์ แต่มองไม่เห็นวาระบบปฏิบัติการสร้าง process ต่อท่อ และรอให้โปรแกรมทำงานเสร็จอย่างไร SeeSh ทำให้ขั้นตอนเหล่านี้มองเห็นได้ หลักการเบื้องหลังเหมือน shell ทั่วไปทุกอย่าง สิ่งที่เพิ่มเข้าไปคือการแสดงผลการทำงาน

2. สิ่งที่ได้

1. SeeSh รันโปรแกรมทุกตัวที่อยู่ใน PATH ได้ และมีคำสั่ง built-in 6 คำสั่ง คือ `cd`, `pwd`, `jobs`, `history`, `help` และ `exit`
2. ผู้ใช้เปลี่ยนทางเข้าออกของข้อมูลด้วย `<`, `>` และ `>>` ได้ทั้งกับโปรแกรมภายนอกและ built-in ต่อหลายโปรแกรมด้วย pipe ได้หลายชั้น และใช้ pipe ร่วมกับ redirection ในคำสั่งเดียวกันได้
3. คำสั่งที่ลงท้ายด้วย `&` จะทำงานเบื้องหลัง เมื่อเสร็จ shell แจ้ง Done และไม่มี zombie process เหลือ ส่วน `Ctrl+C` หยุดเฉพาะโปรแกรมที่รันอยู่ ตัว shell ไม่ปิด
4. นอกจากนี้ยังมีหน้าเว็บ Step Mode ที่แสดงการทำงานของแต่ละคำสั่งที่ละขั้น และบอกได้ว่าคำสั่งที่ผิดพลาดล้มที่ขั้นไหนเพราะอะไร
5. สิ่งที่ไม่ได้ทำ ได้แก่ ตัวแปร, wildcard (`*.c`), `~`, `;` && `||`, scripting, `fg/bg` และ pipe ที่ทำงานเบื้องหลัง

3. การทำงานของระบบ

ระบบมี 3 ส่วนที่เป็นโปรแกรมแยกกัน ดังภาพที่ 1



ภาพที่ 1 โครงสร้างของระบบ SeeSh

1. seesh คือตัว shell ทำหน้าที่รับคำสั่งจากผู้ใช้และรันจริงผ่าน kernel ระหว่างรัน shell จะเขียนข้อมูลของแต่ละขั้น เช่น เรียก fork ได้ PID อะไร เป็นข้อความ JSON ลงใน FIFO
2. seesh-viewer คือเว็บเซิร์ฟเวอร์ภาษา C ที่อ่านข้อมูลจาก FIFO แล้วส่งต่อให้ browser แบบ real-time ด้วย Server-Sent Events (WHATWG, n.d.)
3. หน้าเว็บ นำข้อมูลที่ได้อามาเป็นกล่อง process ท่อ และไฟล์ พร้อมคำอธิบายของแต่ละขั้น shell ไม่ต้องรอหน้าเว็บ ถ้าไม่ได้เปิดหน้าเว็บหรือปิดไปกลางคัน shell ก็ยังใช้งานได้ปกติ

ตัวอย่างเบื้องหลังคำสั่ง ls | wc -l

```
shell  pipe()          สร้างท่อ: fd[3] = ปลายอ่าน, fd[4] = ปลายเขียน
shell  fork()          สร้างลูก A
      ลูก A dup2(4, STDOUT) ให้ทางออกของ ls ลงท่อ
      ลูก A execvp("ls")
shell  fork()          สร้างลูก B
      ลูก B dup2(3, STDIN)  ให้ wc อ่านจากท่อ
      ลูก B execvp("wc")
shell  close(3), close(4) ปิดปลายท่อ (ถ้าไม่ปิด wc จะรอไม่จบ)
shell  waitpid(A), waitpid(B)
```

4. System call ที่ใช้

รายละเอียดของแต่ละฟังก์ชันดูได้จาก Linux manual pages ส่วนที่ 2 (Linux man-pages project, n.d.)

ตารางที่ 1 System call ที่ใช้ใน SeeSh

กลุ่ม	System call	ใช้ทำอะไร
Process	fork execvp waitpid	สร้าง process ลูก ให้ลูกกลายเป็นโปรแกรมที่สั่งและรอให้จบ
Directory	chdir getcwd	คำสั่ง cd และ pwd
File I/O	open close dup dup2	redirection และการต่อท่อเข้ากับ stdin/stdout
Signal	sigaction signal setpgid	Ctrl+C, SIGCHLD เมื่อลูกจบ, แยกงานเบื้องหลังออกจาก Ctrl+C
IPC	pipe mkfifo	ท่อระหว่างโปรแกรม และ FIFO ระหว่าง shell กับเว็บเซิร์ฟเวอร์
Network	socket bind listen accept poll	เว็บเซิร์ฟเวอร์รอหลายการเชื่อมต่อพร้อมกันโดยไม่ใช้ thread

5. ผลการทำงาน

5.1 คำสั่งพื้นฐาน, redirection และ pipe

```
○ seesh:~/Documents/OSminiProject/seesh$ make run
cc -Wall -Wextra -std=gnu11 -g src/builtins.c src/events.c
src/executor.c src/history.c src/jobs.c src/main.c src/pa
rser.c src/pipeline.c src/redirect.c src/signals.c -o sees
h
./seesh
Welcome to SeeSh v0.1 - type 'exit' to quit
seesh:~/Documents/OSminiProject/seesh$ pwd
/Users/arnataree/Documents/OSminiProject/seesh
seesh:~/Documents/OSminiProject/seesh$ ls
Makefile                seesh-viewer.dSYM
README.md                seesh.dSYM
seesh                    src
seesh-viewer             viewer
seesh:~/Documents/OSminiProject/seesh$ echo hi
hi
seesh:~/Documents/OSminiProject/seesh$ history
1 pwd
2 ls
3 echo hi
4 history
seesh:~/Documents/OSminiProject/seesh$ history > /tmp/h.tx
t
seesh:~/Documents/OSminiProject/seesh$ cat /tmp/h.txt
1 pwd
2 ls
3 echo hi
4 history
5 history > /tmp/h.txt
seesh:~/Documents/OSminiProject/seesh$ ls |
seesh: syntax error
seesh:~/Documents/OSminiProject/seesh$ history
1 pwd
2 ls
3 echo hi
4 history
5 history > /tmp/h.txt
6 cat /tmp/h.txt
7 ls |
8 history
seesh:~/Documents/OSminiProject/seesh$ help
SeeSh built-in commands:
cd [dir]    เปลี่ยนโฟลเดอร์
pwd         แสดงโฟลเดอร์ปัจจุบัน
jobs        แสดง background job
history     แสดงคำสั่งที่เคยพิมพ์
help        แสดงรายการนี้
exit        ออกจาก shell
seesh:~/Documents/OSminiProject/seesh$ exit
Bye!
seesh:~/Documents/OSminiProject/seesh$ make run
./seesh
Welcome to SeeSh v0.1 - type 'exit' to quit
seesh:~/Documents/OSminiProject/seesh$ history
1 history
seesh:~/Documents/OSminiProject/seesh$
```

ภาพที่ 2 คำสั่งพื้นฐาน, history, syntax error และ help

```

arnataree@MacBook-Air-khxng-ArnatAree seesh % make run
cc -Wall -Wextra -std=gnu11 -g src/builtins.c src/executor.c src/main.c src/pa
ser.c src/pipeline.c src/redirect.c -o seesh
./seesh
Welcome to SeeSh v0.1 - type 'exit' to quit
seesh:/Users/arnataree/Documents/OSminiProject/seesh$ cd /tmp
seesh:/private/tmp$ echo Hello > hello.txt
seesh:/private/tmp$ cat hello.txt
Hello
seesh:/private/tmp$ echo World >> hello.txt
seesh:/private/tmp$ cat hello.txt
Hello
World
seesh:/private/tmp$ wc -l < hello.txt
2
seesh:/private/tmp$ ls | wc -l
4
seesh:/private/tmp$ ls | grep a | wc -l
3
seesh:/private/tmp$ cat < hello.txt | sort -r >
seesh: syntax error
seesh:/private/tmp$ cat < hello.txt | sort -r > sorted.txt
seesh:/private/tmp$ cat sorted.txt
World
Hello
seesh:/private/tmp$ cat < ไม่มีไฟล์.txt
ไม่มีไฟล์.txt: No such file or directory
seesh:/private/tmp$

```

ภาพที่ 3 redirection และ pipe หลายชั้น

5.2 Ctrl+C และงานเบื้องหลัง

กด Ctrl+C แล้ว sleep หยุดแต่ shell ยังอยู่ งานที่สั่งด้วย & แจ้ง Done เมื่อเสร็จ และ ps ไม่พบ process สถานะ Z (zombie) ซึ่งเป็นสถานะที่เรียนในการทดลองที่ 3 (การทดลองที่ 3, ม.ป.ป.)

```

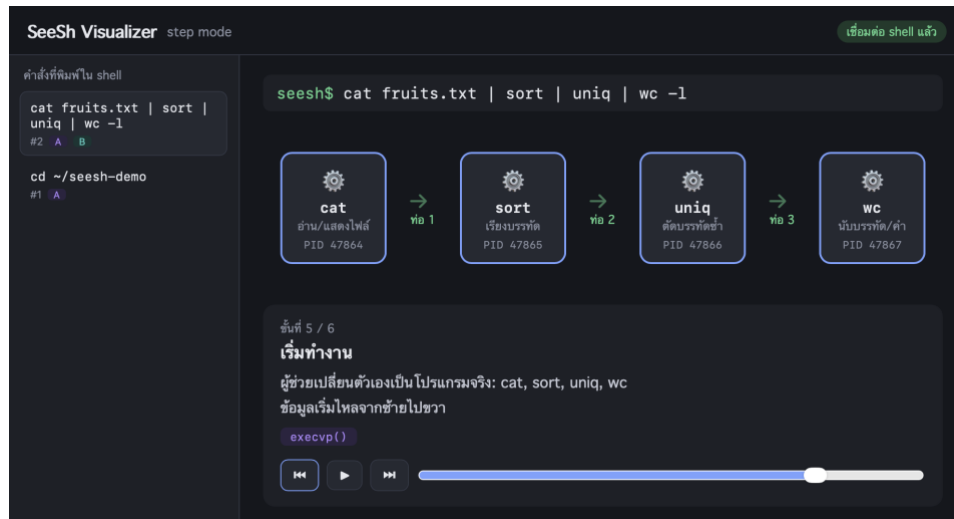
arnataree@MacBook-Air-khxng-ArnatAree seesh % make run
cc -Wall -Wextra -std=gnu11 -g src/builtins.c src/executor.c src/jobs.c src/mai
n.c src/parser.c src/pipeline.c src/redirect.c src/signals.c -o seesh
./seesh
Welcome to SeeSh v0.1 - type 'exit' to quit
seesh:/Users/arnataree/Documents/OSminiProject/seesh$ sleep 30
^C
seesh:/Users/arnataree/Documents/OSminiProject/seesh$ sleep 20 &
[1] 66575
seesh:/Users/arnataree/Documents/OSminiProject/seesh$ jobs
[1]  Running   sleep 20 &
[1]  Done     sleep 20 &
seesh:/Users/arnataree/Documents/OSminiProject/seesh$ sleep 20 &
[1] 66837
seesh:/Users/arnataree/Documents/OSminiProject/seesh$ jobs
[1]  Running   sleep 20 &
seesh:/Users/arnataree/Documents/OSminiProject/seesh$ sleep 3 &
[2] 67005
[1]  Done     sleep 20 &
seesh:/Users/arnataree/Documents/OSminiProject/seesh$
[2]  Done     sleep 3 &
seesh:/Users/arnataree/Documents/OSminiProject/seesh$ ps -o pid,stat,comm
  PID STAT COMM
65678 Ss  /bin/zsh
66173 S+  /Library/Developer/CommandLineTools/usr/bin/make
66198 S+  ./seesh
seesh:/Users/arnataree/Documents/OSminiProject/seesh$ ^C

```

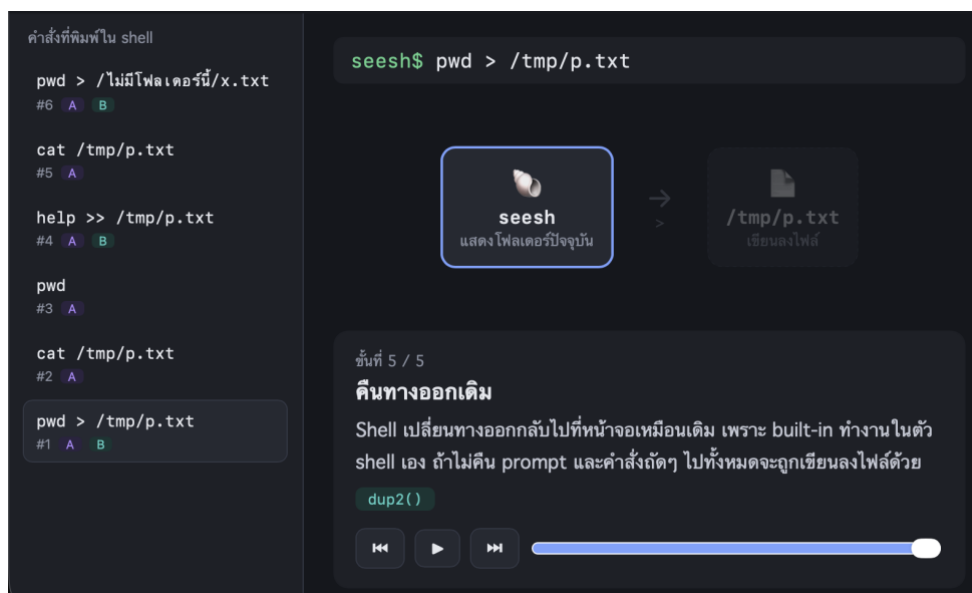
ภาพที่ 4 Ctrl+C, งานเบื้องหลัง, jobs และตรวจ zombie ด้วย ps

5.3 หน้าเว็บ Step Mode

เลือกคำสั่งทางซ้าย แล้วกดดูทีละขั้นหรือเล่นอัตโนมัติ แต่ละขั้นมีป้ายชื่อ system call กำกับ



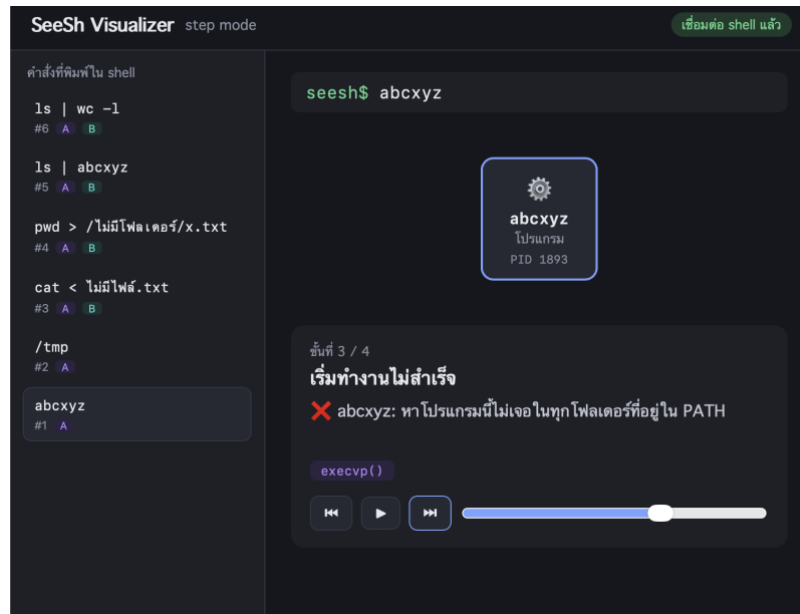
ภาพที่ 5 คำสั่ง cat fruits.txt | sort | uniq | wc -l: process ลูก 4 ตัวพร้อม PID จริง ต่อกันด้วยท่อ 3 เส้น



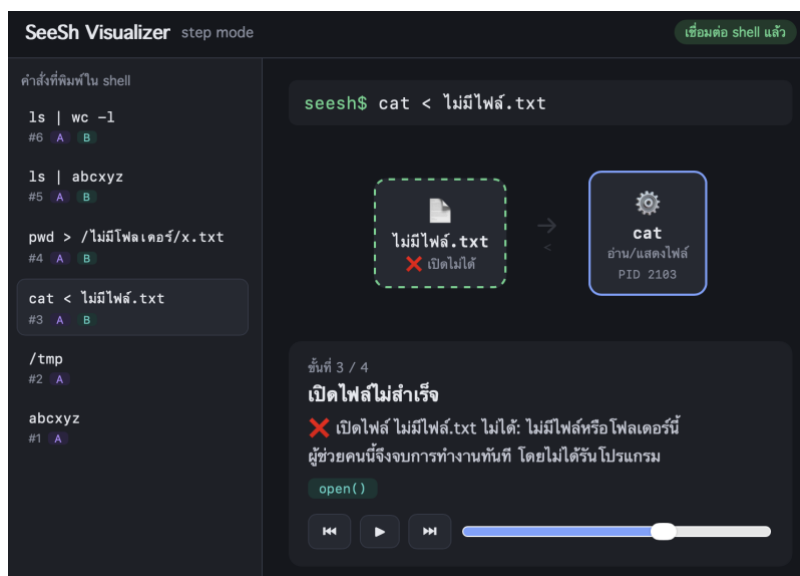
ภาพที่ 6 pwd > /tmp/p.txt: built-in ต้องคืนทางออกเดิมของ shell หลังทำงานเสร็จ

5.4 การอธิบายข้อผิดพลาด

ถ้าคำสั่งล้มเหลว หน้าเว็บบอกขั้นที่ล้มและเหตุผล exit status ใช้ค่าเดียวกับ bash คือ 127 เมื่อหาโปรแกรมไม่เจอ และ 126 เมื่อรันไม่ได้ (Free Software Foundation, 2022)



ภาพที่ 7 abcxyz ล้มเหลวที่ขั้น execvp เพราะหาโปรแกรมไม่เจอ



ภาพที่ 8 cat < ไม่มีไฟล์.txt ล้มเหลวที่ขั้น open และ cat ไม่เคยถูกรัน

5.5 ปิดหน้าเว็บแล้ว shell ยังทำงานได้

```
seesh:/Users/arnataree/D
/seesh$ make viewer
cc -Wall -Wextra -std=gn
u11 -g viewer/server.c -
o seesh-viewer
./seesh-viewer
SeeSh Viewer: http://loc
alhost:8080 (กด Ctrl+C
เพื่อปิด)
^C

make[2]: *** [viewer] In
terrupt: 2
seesh:/Users/arnataree/D
ocuments/OSminiProject/s
eesh$
```

```
Bye!
❖arnataree@MacBook-Air-khxng-Arnataree seesh % make
run
./seesh
Welcome to SeeSh v0.1 - type 'exit' to quit
seesh:/Users/arnataree/Documents/OSminiProject/see
sh$ pwd
/Users/arnataree/Documents/OSminiProject/seesh
seesh:/Users/arnataree/Documents/OSminiProject/see
sh$ pwd
/Users/arnataree/Documents/OSminiProject/seesh
seesh:/Users/arnataree/Documents/OSminiProject/see
sh$ pwd
/Users/arnataree/Documents/OSminiProject/seesh
seesh:/Users/arnataree/Documents/OSminiProject/see
sh$
```

ภาพที่ 9 ซ้าย: ปิดเว็บเซิร์ฟเวอร์ · ขวา: seesh ยังรับคำสั่งต่อได้

6. การทดสอบ

make test ป้อนคำสั่งชุดเดียวกันเข้าทั้ง SeeSh และ bash แล้วเทียบผล ถ้าคำสั่งไหนค้างเกิน 5 วินาทีนับว่าไม่ผ่าน ส่วน Ctrl+C และหน้าเว็บทดสอบด้วยมือ ผลคือ ผ่าน 32 จาก 32 กรณี ทั้งบน macOS, Linux (GitHub Codespaces) และ Windows ผ่าน WSL (Microsoft, n.d.) ซึ่งติดตั้งไว้ตั้งแต่ใบงานปฏิบัติการที่ 1 (ใบงานปฏิบัติการที่ 1, ม.ป.ป.)

ตารางที่ 2 ชุดทดสอบอัตโนมัติ

ชุดทดสอบ	จำนวน	ทดสอบอะไร
test_core.sh	13	คำสั่งพื้นฐาน, built-in, หาโปรแกรมไม่เจอ, syntax error, history
test_io.sh	14	redirection, pipe 2-4 ชั้น, ข้อมูล 100,000 บรรทัดผ่านท่อ, built-in + redirect
test_signals.sh	5	งานเบื้องหลัง, jobs, แจ้ง Done, ไม่มี zombie

```
seesh:~/Documents/OSminiProject/seesh$ make test
bash tests/run_all.sh
```

```
== Core: คำสั่งพื้นฐานและ built-in ==
```

- ✓ echo
- ✓ ขอความในเครื่องหมายคำพูด
- ✓ ls
- ✓ cd เข้าโฟลเดอร์ย่อย
- ✓ cd ไม่ใส่อะไร = กลับ home
- ✓ exit หยุดทันที
- ✓ บรรทัดว่างไม่ error
- ✓ หาโปรแกรมไม่เจอ
- ✓ เจอแต่รันไม่ได้
- ✓ syntax error
- ✓ shell ทำงานต่อหลัง error
- ✓ help มี history
- ✓ history บันทึกคำสั่ง

ผ่าน 13 / 13

```
== I/O: redirect และ pipe ==
```

- ✓ > เขียนทับ
- ✓ >> เขียนต่อท้าย
- ✓ < อ่านจากไฟล์
- ✓ pipe 2 ชั้น
- ✓ pipe 3 ชั้น
- ✓ pipe 4 ชั้น
- ✓ pipe + redirect
- ✓ ข้อมูลเยอะผ่านท่อ (ไม่ค้าง)
- ✓ โปรแกรมอ่านไม่หมด (ไม่ค้าง)
- ✓ built-in + redirect
- ✓ คั่นทางออกหลัง built-in
- ✓ history + redirect
- ✓ เปิดไฟล์ไม่ได้
- ✓ เขียนไฟล์ในโฟลเดอร์ที่ไม่มี

ผ่าน 14 / 14

```
== Signals: background job ==
```

- ✓ & แจกหมายเลข job
- ✓ jobs แสดงงานที่ยังทำอยู่
- ✓ แจก Done เมื่องานจบ
- ✓ ไม่มี zombie หลงเหลือ
- ✓ shell ไม่รอกงานเบื้องหลัง

ผ่าน 5 / 5

ผ่านทั้งหมด ✓

```
seesh:~/Documents/OSminiProject/seesh$
```

ภาพที่ 10 make test บน macOS

```

@Arnat-Aree → /workspaces/seesh (main) $ make test
cc -Wall -Wextra -std-gnu11 -g src/builtins.c src/events.c src/executor.c src/h
istory.c src/jobs.c src/main.c src/parser.c src/pipeline.c src/redirect.c src/s
ignals.c -o seesh
bash tests/run_all.sh

== Core: คำสั่งพื้นฐานและ built-in ==
✓ echo
✓ ข้อความในเครื่องหมายคำพูด
✓ ls
✓ cd เข้าโฟลเดอร์ย่อย
✓ cd ไม่ใส่อะไร = กลับ home
✓ exit หยุดทันที
✓ บรรทัดว่างไม่ error
✓ หาโปรแกรมไม่เจอ
✓ เจอแต่รันไม่ได้
✓ syntax error
✓ shell ทำงานต่อหลัง error
✓ help มี history
✓ history บันทึกคำสั่ง
ผ่าน 13 / 13

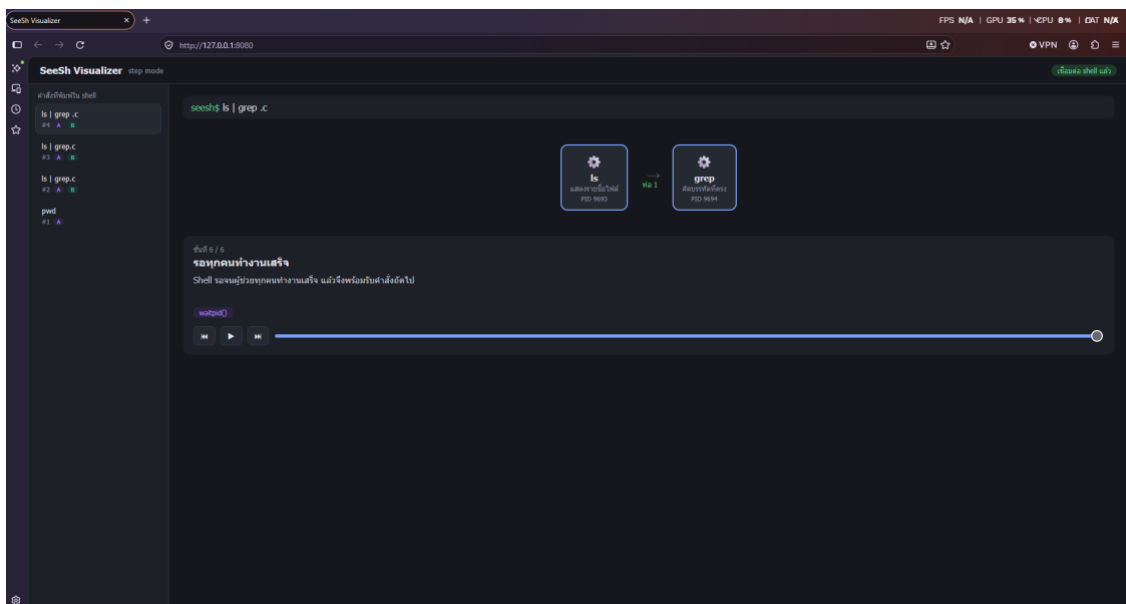
== I/O: redirect และ pipe ==
✓ > เขียนทับ
✓ >> เขียนต่อท้าย
✓ < อ่านจากไฟล์
✓ pipe 2 ชั้น
✓ pipe 3 ชั้น
✓ pipe 4 ชั้น
✓ pipe + redirect
✓ ข้อมูลเยอะผ่านท่อ (ไม่ค้าง)
✓ โปรแกรมอ่านไม่หมด (ไม่ค้าง)
✓ built-in + redirect
✓ ค้นหาออกหลัง built-in
✓ history + redirect
✓ เปิดไฟล์ไม่ได้
✓ เขียนไฟล์ในโฟลเดอร์ที่ไม่มี
ผ่าน 14 / 14

== Signals: background job ==
✓ & แจ้งหมายเลข job
✓ jobs แสดงงานที่ยังทำอยู่
✓ แฉง Done เมื่องานจบ
✓ ไม่มี zombie หลงเหลือ
✓ shell ไม่รอจนจบเบื้องหลัง
ผ่าน 5 / 5

ผ่านทั้งหมด ✓
@Arnat-Aree → /workspaces/seesh (main) $

```

ภาพที่ 11 make test บน Linux (Codespaces)



ภาพที่ 12 ใช้งานบน Windows: shell รันใน WSL และ browser บน Windows แสดงผล

7. ปัญหาที่เจอและวิธีแก้

ตารางที่ 3 ปัญหาที่เจอระหว่างพัฒนา

ปัญหา	สาเหตุ	วิธีแก้
Pipe ค้าง	ปิดปลายท่อที่ไม่ใช้ไม่ครบ ฝั่งอ่านเลยรอไม่จบ	ปิดทุกปลายที่ไม่ใช้ทั้งในแม่และลูก + เทสต์จับเวลา 5 วินาที
Ctrl+C แล้ว shell ปิด	SIGINT ส่งไปถึง shell ด้วย	ติดตั้ง handler ด้วย sigaction
pwd > file ยังพิมพ์ลงจอ	built-in ไม่ได้ fork จึงไม่ผ่าน redirect	จำ stdout ด้วย dup แล้วคืนค่าด้วย dup2
ปิด viewer แล้ว shell ตาย	เขียนลงท่อที่ไม่มีคนอ่านได้ SIGPIPE	ไม่สนใจ SIGPIPE และเปิด FIFO แบบ non-blocking
หน้าเว็บบอกว่าคำสั่งผิดรันสำเร็จ	ส่งข้อมูล exec ก่อนเรียก execvp	เพิ่มข้อมูล exec_fail และ open_fail พร้อม errno
Git บน Windows ขึ้นว่าแก้ทุกไฟล์	Windows ใช้ CRLF แต่ Mac/Linux ใช้ LF	เพิ่ม .gitattributes บังคับ LF

8. ข้อจำกัด

ยังใช้ built-in ใน pipe ไม่ได้ เช่น history | grep ls และยังไม่มี fg, bg หรือ Ctrl+Z สำหรับจัดการงานหน้าเว็บเปิดได้จากเครื่องเดียวกันเท่านั้น และเนื่องจากทุก shell ใช้ FIFO เดียวกัน ถ้ารัน make test ระหว่างเปิดหน้าเว็บ ข้อมูลจากหลาย shell จะปนกัน

9. สรุป

SeeSh เป็น shell ภาษา C ที่ใช้ system call จริง รองรับคำสั่งพื้นฐาน, redirection, pipe, งานเบื้องหลัง และ Ctrl+C พร้อมหน้าเว็บที่แสดงการทำงานที่ละเอียดและอธิบายข้อผิดพลาดได้ ทดสอบผ่าน 32 จาก 32 กรณีบน macOS, Linux และ Windows

ภาคผนวกวิธีติดตั้งและใช้งาน

ส่วนนี้อธิบายตั้งแต่เตรียมเครื่องจนรันได้ ใช้งานได้กับ macOS, Linux และ Windows (ผ่าน WSL) คำสั่งทั้งหมดพิมพ์ใน Terminal

ก. เตรียมเครื่อง

SeeSh ต้องใช้ตัวคอมไพล์ภาษา C (gcc หรือ clang), make และ git บน macOS ติดตั้งด้วยคำสั่งแรก บน Linux หรือ WSL ใช้คำสั่งที่สอง

```
xcode-select --install          # macOS
sudo apt install -y build-essential git  # Linux / WSL
```

ผู้ใช้ Windows ต้องติดตั้ง WSL ก่อน (Microsoft, n.d.) แล้วทำทุกขั้นตอนต่อจากนี้ใน Terminal ของ WSL

ข. ดาวน์โหลดและคอมไพล์

```
git clone https://github.com/Arnat-Aree/seesh.git
cd seesh
make
```

make จะสร้างโปรแกรม 2 ตัวในโฟลเดอร์ คือ seesh (shell) และ seesh-viewer (เว็บเซิร์ฟเวอร์) ถ้าไม่มีข้อความ error แสดงว่าคอมไพล์สำเร็จ

สำหรับ WSL ให้ clone ไว้ใน home ของ Linux (เช่น /home/ชื่อผู้ใช้) ไม่ใช่ใน /mnt/c/... เพราะ Git ผี Windows จะเปลี่ยนการขึ้นบรรทัดของไฟล์เป็น CRLF ทำให้สคริปต์ทดสอบรันไม่ได้

ค. รัน shell พร้อมหน้าเว็บ

เปิด Terminal 2 หน้าต่าง และ cd เข้าโฟลเดอร์ seesh ทั้งสองหน้าต่าง หน้าต่างแรกใช้รันเว็บเซิร์ฟเวอร์

```
make viewer
```

จากนั้นเปิด browser ไปที่ <http://localhost:8080> บน Windows เปิดด้วย browser ของ Windows ได้เลย แล้วกลับมาที่หน้าต่างที่สองเพื่อเปิด shell

```
make run
```

เมื่อเห็น prompt ของ SeeSh ก็พิมพ์คำสั่งได้เหมือน shell ทั่วไป ถ้าไม่ต้องการดูหน้าเว็บ ข้ามหน้าต่างแรกไปรัน make run อย่างเดียวก็ได้

ง. ลองคำสั่งและดูผลบนหน้าเว็บ

ตัวอย่างคำสั่งที่ครอบคลุมทุกความสามารถ

```
pwd                # built-in
echo Hello > /tmp/hello.txt    # เขียนผลลงไฟล์
cat < /tmp/hello.txt | sort    # อ่านจากไฟล์แล้วส่งต่อด้วย pipe
ls | grep c | wc -l           # pipe 3 ชั้น
sleep 3 &              # งานเบื้องหลัง
jobs                      # ดูงานเบื้องหลัง
history                 # ดูคำสั่งที่เคยพิมพ์
abxcyz                  # คำสั่งที่ไม่มีอยู่จริง
```

ทุกคำสั่งที่พิมพ์จะขึ้นเป็นรายการทางซ้ายของหน้าเว็บ คลิกเลือกคำสั่งที่ต้องการ แล้วกดปุ่มถัดไปเพื่อดูทีละชั้น หรือปุ่มเล่นเพื่อให้เล่นอัตโนมัติ และลากแถบเลื่อนเพื่อย้อนไปชั้นใดก็ได้ แต่ละชั้นมีชื่อ system call และคำอธิบายภาษาไทยกำกับ

ถ้าต้องการลอง Ctrl+C ให้รัน sleep 30 แล้วกด Ctrl+C โปรแกรม sleep จะหยุดและขึ้น prompt ใหม่ โดยที่ shell ยังเปิดอยู่

จ. รันชุดทดสอบ

```
make test
```

คำสั่งนี้รันชุดทดสอบทั้ง 3 ชุด (ตารางที่ 2) โดยเทียบผลของ SeeSh กับ bash ถ้าผ่านทั้งหมดจะขึ้นว่า “ผ่านทั้งหมด ✓” ควรปิดหน้าเว็บก่อนรัน เพราะข้อมูลจากการทดสอบจะไปบนบนหน้าเว็บ

ฉ. ปิดโปรแกรม

พิมพ์ exit เพื่อออกจาก SeeSh และกด Ctrl+C ในหน้าต่างของ seesh-viewer เพื่อปิดเว็บเซิร์ฟเวอร์ ถ้าต้องการลบไฟล์ที่คอมไพล์ไว้ใช้ make clean

ช. ข้อควรระวัง

SeeSh ไม่แปลง ~ เป็น home directory ให้ใช้ cd เฉยๆ เพื่อกลับ home แทน ไม่ควรกด Ctrl+Z เพราะยังไม่มี fg สำหรับเรียกงานกลับมา และ SeeSh ไม่รองรับ *, ;, &&, || หรือตัวแปรอย่าง \$HOME คำสั่งเหล่านี้จะถูกส่งให้โปรแกรมตามตัวอักษร

เอกสารอ้างอิง

Free Software Foundation. (2022). *GNU Bash reference manual* (Version 5.2).

<https://www.gnu.org/software/bash/manual/>

Linux man-pages project. (n.d.). *Linux manual pages: Section 2, system calls*.

https://man7.org/linux/man-pages/dir_section_2.html

Microsoft. (n.d.). *What is the Windows Subsystem for Linux?* Microsoft Learn.

<https://learn.microsoft.com/en-us/windows/wsl/about>

The Open Group. (2018). *The Open Group base specifications issue 7, 2018 edition (IEEE Std 1003.1-2017)*. <https://pubs.opengroup.org/onlinepubs/9699919799/>

WHATWG. (n.d.). *HTML living standard: Server-sent events*.

<https://html.spec.whatwg.org/multipage/server-sent-events.html>

การทดลองที่ 3: สถานะของกระบวนการและ Process Control Block (PCB). (ม.ป.ป.). [ใบงานปฏิบัติการ].

รายวิชา Operating Systems and System Calls Programming มหาวิทยาลัยขอนแก่น.

ใบงานปฏิบัติการที่ 1: การใช้งานพื้นฐานของระบบปฏิบัติการลินุกซ์ผ่าน WSL พร้อมการตรวจสอบ System Calls. (ม.ป.ป.). [ใบงานปฏิบัติการ]. รายวิชา Operating Systems and System Calls Programming มหาวิทยาลัยขอนแก่น.